



Unversidade de Santiago

Departamento de Ciências da Saúde, Ambiente e Tecnologia

Disciplina: Teste de Qualidade de Software

Projeto Final: Banco Digital Kriolu

Banco Digital Kriolu
Autenticação Segura

Utilizador
Introduza o utilizador

Palavra-passe
Introduza a palavra-passe

☒ Lembrar utilizador

☐ Confiar neste dispositivo

Entrar

Entrar com Biometria

Elementos de Grupo :

Docente: Aldina Semedo

- ❖ Isberto Varela (a7367)
- ❖ Rafael da Moura (7462)
- ❖ Kleidmilson Alves (a7045)
- ❖ Ruth de Melo(a7006)

Julho de 2026.

Contents

3.Introdução.....	3
3.1Contexto do Negócio.....	3
3.2.Objetivo do Projeto.....	3
3.3 Funcionalidades Principais do Sistema	4
4. ENGENHARIA DE REQUISITOS.....	5
4.1 User Stories	5
4.2 Critérios de Aceitação	6
6.FLUXO PRINCIPAL DO SISTEMA.....	9
6.1Diagrama do Fluxo Principal.....	10
6.2Tecnologias Envolvidas no Fluxo	11
7.PLANO DE TESTES	12
7.1 Objetivos dos Testes.....	12
7.2 Estratégia de Testes	12
7.3 Casos de Teste Implementados	13
11.4 Ambiente de Testes.....	14
11.5 Processo de Execução dos Testes.....	14
8.BDD	15
8.1 Cenários Gherkin	15
9.CASOS DE TESTE.....	17
10.TESTES API.....	19
11.GESTÃO DE DEFEITOS.....	21
12.AUTOMAÇÃO DE TESTES	23
13.Pipeline CI/CD.....	32
14.MÉTRICAS DE QUALIDADE.....	33
15. CONCLUSÃO	35
17. REFERÊNCIAS BIBLIOGRÁFICAS.....	36

3.Introdução

3.1Contexto do Negócio

A transformação digital tem vindo a revolucionar o setor financeiro, proporcionando aos clientes maior comodidade, rapidez e acessibilidade na realização de operações bancárias. Com o crescimento dos serviços digitais, aumentou também a necessidade de garantir elevados níveis de segurança, privacidade e fiabilidade, tornando indispensável a implementação de mecanismos robustos de autenticação e proteção dos dados dos utilizadores.

Neste contexto, foi desenvolvido o **Banco Digital Kriolu**, uma aplicação web que simula o funcionamento de um banco digital moderno. O sistema foi concebido para oferecer uma experiência de utilização simples e intuitiva, sem comprometer os requisitos de segurança exigidos em ambientes financeiros. Entre as suas principais funcionalidades destacam-se a autenticação por nome de utilizador e palavra-passe, a autenticação multifator (OTP), o login biométrico, o registo de dispositivos confiáveis, a gestão do histórico de acessos, o envio de alertas por Email e SMS e a gestão segura das sessões dos utilizadores.

Para além da implementação funcional, o projeto teve como principal foco a aplicação das boas práticas de **Testes e Qualidade de Software**, assegurando que todas as funcionalidades desenvolvidas fossem devidamente verificadas e validadas antes da disponibilização da aplicação.

3.2.Objetivo do Projeto

O principal objetivo deste projeto consiste no desenvolvimento e validação de uma aplicação bancária digital segura, aplicando metodologias e ferramentas modernas de testes de software ao longo de todo o ciclo de desenvolvimento.

Pretendeu-se demonstrar a importância da qualidade do software através da definição de requisitos, elaboração de casos de teste, automatização de testes funcionais e de **API**, análise da qualidade segundo a norma **ISO/IEC 25010** e implementação de um processo de Integração Contínua e Entrega Contínua (**CI/CD**).

Especificamente, o projeto teve como objetivos:

- Desenvolver uma aplicação bancária funcional e segura;
- Implementar mecanismos de autenticação baseados em múltiplos fatores (**MFA**);
- Automatizar testes funcionais utilizando **Playwright**;
- Validar os serviços da **API REST** através de testes automatizados;
- Garantir a qualidade do software com base em critérios internacionais;

- Implementar um pipeline de **CI/CD** utilizando **GitHub Actions** e **Render**;
- Reduzir erros durante o desenvolvimento através da execução automática dos testes a cada alteração do código.

Desta forma, procurou-se aproximar o projeto de um ambiente real de desenvolvimento profissional, onde a qualidade, a automação e a entrega contínua são elementos fundamentais para garantir software fiável e de elevada qualidade.

3.3 Funcionalidades Principais do Sistema

O **Banco Digital Kriolu** disponibiliza um conjunto de funcionalidades que simulam operações essenciais de um sistema bancário digital moderno, destacando-se:

- Autenticação através de nome de utilizador e palavra-passe;
- Autenticação Multifator (**OTP**) para reforço da segurança;
- Login biométrico;
- Registo e reconhecimento de dispositivos confiáveis;
- Dashboard com informações do utilizador;
- Consulta do histórico de acessos e operações;
- Envio de alertas por Email e SMS após autenticação;
- Encerramento seguro da sessão (Logout);
- **API REST** para comunicação entre o frontend e o backend;
- Automatização de **testes funcionais** e de **API**;
- Integração Contínua (**CI**) com **GitHub Actions**;
- Entrega Contínua (**CD**) com **Render**, permitindo a publicação automática da aplicação após a validação dos testes.

Estas funcionalidades foram desenvolvidas com o objetivo de proporcionar uma aplicação segura, eficiente e alinhada com as boas práticas de engenharia de software, permitindo demonstrar, de forma prática, a aplicação dos conceitos estudados na unidade curricular de **Testes e Qualidade de Software**.

4. ENGENHARIA DE REQUISITOS

4.1 User Stories

User Story 1 – Login no sistema

Como cliente do Banco Digital Kriolu,
quero iniciar sessão com o meu utilizador e palavra-passe,
para aceder à minha conta bancária de forma segura.

User Story 2 – Autenticação com OTP

Como cliente autenticado,
quero receber um código OTP por SMS ou e-mail após o login,
para confirmar a minha identidade antes de aceder à conta.

User Story 3 – Alteração da palavra-passe

Como cliente do Banco Digital Kriolu,
quero alterar a minha palavra-passe,
para manter a minha conta protegida.

User Story 4 – Escolha do método de OTP

Como cliente,
quero escolher receber o código OTP por e-mail ou SMS,
para utilizar o método mais conveniente.

User Story 5 – Reconhecimento de dispositivo confiável

Como cliente,
quero registar um dispositivo como confiável,

para tornar os próximos acessos mais rápidos e seguros, sem comprometer a proteção da conta.

User Story 6 – Autenticação biométrica

Como cliente que utiliza um smartphone compatível,
quero entrar na aplicação utilizando a minha impressão digital,
para aceder à conta de forma mais rápida e segura.

4.2 Critérios de Aceitação

1. Login do utilizador

Dado que o utilizador possui uma conta válida, quando introduzir o nome de utilizador e a palavra-passe corretos, então o sistema deve permitir o acesso à etapa de autenticação em dois fatores (2FA).

2. Validação da palavra-passe

O sistema deve rejeitar palavras-passe incorretas e apresentar uma mensagem de erro, sem revelar informações que possam comprometer a segurança.

3. Envio do código OTP

Após o login com sucesso, o sistema deve enviar automaticamente um código OTP para o número de telefone ou e-mail registado pelo utilizador.

4. Validação do OTP

O acesso à conta só deve ser permitido quando o utilizador introduzir um código OTP válido e dentro do prazo de validade.

5. Recuperação da conta

O utilizador deve dirigir ao banco Digital Kriolu para que a sua identidade seja validada por um colaborador autorizado então a banco deve permitir a redefinição da palavra-passe e reativar o acesso à conta.

6. Tipos de acessos por OTP

O utilizador deve escolher entre receber o OTP por SMS ou e-mail antes do envio do código.

7. Reconhecimento de dispositivo confiável

Depois de um dispositivo ser marcado como confiável, o sistema deve reconhecê-lo nos acessos seguintes, mantendo as regras de segurança definidas.

8. Autenticação biométrica

Se o dispositivo suportar autenticação biométrica, o utilizador deve poder iniciar a sessão utilizando impressão digital de forma segura.

5. ANÁLISE DA QUALIDADE (ISO/IEC 25010)

A norma ISO/IEC 25010 define características que ajudam a avaliar a qualidade de um sistema de software. No caso do Banco Digital Kriolu, algumas características são mais importantes devido ao elevado número de tentativas de fraude e aos problemas de segurança registados.

1. Segurança

A segurança é o atributo mais importante deste projeto, porque o sistema lida com informações bancárias e dados pessoais dos clientes. Como será implementada a autenticação em dois fatores (2FA), é essencial proteger as contas contra acessos não autorizados, roubo de credenciais, ataques de força bruta e outras tentativas de fraude.

2. Fiabilidade

O sistema deve funcionar corretamente em todos os momentos. O login, a validação da password, o envio e a confirmação do código OTP devem ocorrer sem falhas, garantindo que os utilizadores conseguem aceder às suas contas com confiança.

3. Usabilidade

O processo de autenticação deve ser simples e fácil de utilizar. Os utilizadores devem conseguir fazer login, receber o código OTP e concluir a autenticação sem dificuldades, mesmo que não tenham muitos conhecimentos de tecnologia.

4. Adequação Funcional

O sistema deve cumprir todos os requisitos definidos pelo banco. As funcionalidades como login, recuperação de conta, histórico de acessos, gestão de sessões, autenticação biométrica e reconhecimento de dispositivos confiáveis devem funcionar conforme o esperado.

5. Eficiência de Desempenho

As operações do sistema devem ser rápidas. O envio do código OTP, a autenticação e a validação das credenciais não devem demorar demasiado tempo, proporcionando uma boa experiência ao utilizador.

6. Compatibilidade

O sistema deve funcionar corretamente em diferentes dispositivos e navegadores, permitindo que os clientes utilizem o banco digital no computador, tablet ou telemóvel sem problemas.

7. Manutenibilidade

O sistema deve ser organizado e fácil de atualizar. Isso facilitará a implementação de novas funcionalidades e correções de erros no futuro, reduzindo o tempo de manutenção.

8. Portabilidade

Embora seja importante, a portabilidade não é uma prioridade neste projeto. Ainda assim, o sistema deve poder ser instalado ou adaptado a diferentes ambientes tecnológicos, caso seja necessário.

Atributo ISO/IEC 25010	Prioridade
Segurança	Muito Alta
Fiabilidade	Alta
Usabilidade	Alta
Adequação Funcional	Alta
Eficiência de Desempenho	Média
Compatibilidade	Média
Manutenibilidade	Média
Portabilidade	Baixa

6.FLUXO PRINCIPAL DO SISTEMA

Descrição do Fluxo Principal

O fluxo principal do **Banco Digital Kriolu** representa o processo completo de autenticação e acesso ao sistema, desde a entrada das credenciais até ao encerramento seguro da sessão. Este fluxo foi concebido com base em mecanismos de autenticação multifator (**MFA**), proporcionando maior proteção aos utilizadores e reduzindo os riscos associados a acessos não autorizados.

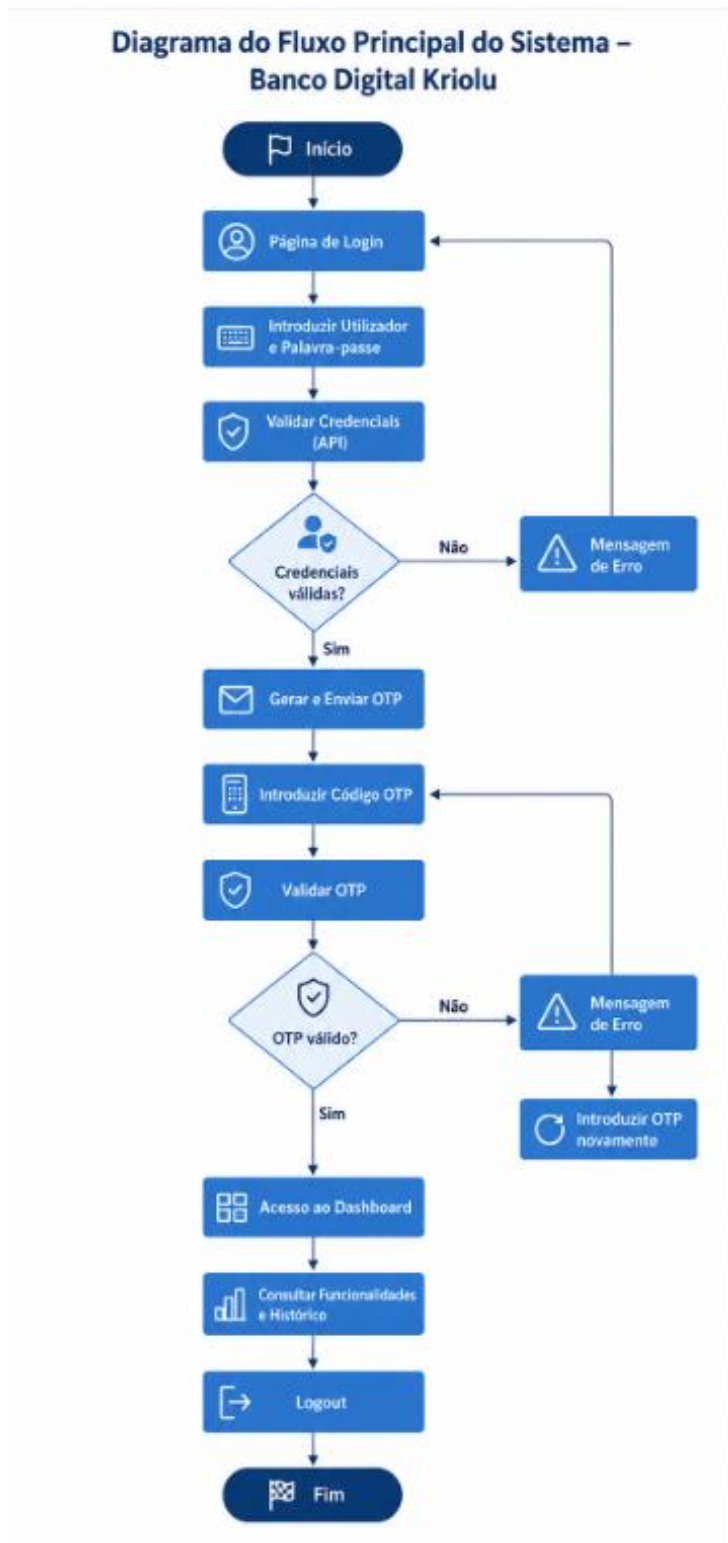
Inicialmente, o utilizador acede à página de login e introduz o seu nome de utilizador e palavra-passe. O sistema valida as credenciais através da **API**. Caso os dados sejam inválidos, é apresentada uma mensagem de erro e o acesso é recusado. Se as credenciais forem válidas, o sistema inicia a segunda etapa de autenticação, enviando um código **OTP (One-Time Password)** para o método de autenticação selecionado pelo utilizador.

Após a receção do código, o utilizador introduz o **OTP** na aplicação. O sistema verifica a sua validade, confirmando se o código corresponde ao gerado e se ainda se encontra dentro do período de expiração. Em caso de sucesso, a autenticação é concluída e o utilizador é redirecionado para o **Dashboard**, onde pode consultar as suas informações, visualizar o histórico de acessos e utilizar as restantes funcionalidades disponibilizadas pela aplicação.

Durante o processo de autenticação, o sistema pode ainda registar o dispositivo como confiável, efetuar autenticação biométrica (quando ativada) e gerar notificações por Email ou SMS, reforçando a segurança e a monitorização das atividades da conta.

Por fim, ao selecionar a opção Logout, a sessão é encerrada de forma segura, sendo igualmente registado o evento no histórico do utilizador.

6.1 Diagrama do Fluxo Principal



6.2 Tecnologias Envolvidas no Fluxo

O fluxo principal do sistema integra diferentes componentes que trabalham em conjunto para garantir um processo de autenticação seguro e eficiente:

- **Frontend** (HTML, CSS e JavaScript): responsável pela interface gráfica e interação com o utilizador.
- **Backend** (Node.js e Express): processa as requisições, valida credenciais, gera códigos OTP e gere as sessões.
- **API REST**: estabelece a comunicação entre o frontend e o backend através de endpoints específicos.
- **Base de Dados** (users.json): armazena as informações dos utilizadores, histórico, configurações de biometria e dispositivos confiáveis.
- **Playwright**: automatiza os testes funcionais e de **API** para validar todas as etapas do fluxo.
- **GitHub Actions**: executa automaticamente os testes sempre que existe uma alteração ao código (Integração Contínua – **CI**).
- **Render**: publica automaticamente a versão atualizada da aplicação após a aprovação dos testes (Entrega Contínua – **CD**).

Este fluxo foi desenvolvido seguindo princípios de **segurança, qualidade e automação**, permitindo demonstrar um processo semelhante ao utilizado em aplicações **bancárias reais**, onde a autenticação multifator e a validação contínua são essenciais para garantir a proteção dos utilizadores e a fiabilidade do sistema.

7. PLANO DE TESTES

O plano de testes do **Banco Digital Kriolu** foi elaborado com o objetivo de garantir a qualidade, fiabilidade, segurança e correto funcionamento da aplicação. A estratégia adotada baseia-se na execução de testes automatizados utilizando o Playwright, integrados num processo de **CI/CD (GitHub Actions + Render)**, permitindo validar automaticamente cada alteração realizada no sistema antes da sua disponibilização.

7.1 Objetivos dos Testes

Os testes têm como principais objetivos:

- Verificar o correto funcionamento das funcionalidades implementadas;
- Validar os requisitos funcionais e de segurança do sistema;
- Garantir que alterações ao código não introduzem novos erros (regressões);
- Confirmar a integração entre o frontend e o backend;
- Assegurar a estabilidade da aplicação durante o processo de integração contínua (**CI**).

7.2 Estratégia de Testes

A estratégia adotada contempla diferentes níveis de validação da aplicação:

Tipo de Teste	Objetivo
Testes Funcionais (UI)	Validar a interação do utilizador com a interface gráfica da aplicação.
Testes de API	Verificar o correto funcionamento dos serviços REST disponibilizados pelo backend.
Testes Positivos	Confirmar que o sistema responde corretamente quando recebe dados válidos.
Testes Negativos	Garantir que o sistema trata corretamente situações de erro e entradas inválidas.
Testes de Regressão	Assegurar que novas alterações não afetam funcionalidades já implementadas.

7.3 Casos de Teste Implementados

O sistema foi validado através de um conjunto abrangente de testes automatizados, cobrindo os principais cenários de utilização.

Aqui estão todos os casos de teste CT para Caso de Teste Funcionais, NT para Casos de Teste negativos e de API:

ID	Caso de Teste	Resultado Esperado
CT-01	Login com credenciais válidas	Login efetuado com sucesso
CT-02	Login com autenticação OTP	Utilizador autenticado
CT-03	Login biométrico	Autenticação biométrica concluída
CT-04	Logout	Sessão terminada corretamente
CT-05	Registo de dispositivo confiável	Dispositivo registado
CT-06	Envio de alerta por Email	Email de alerta enviado
CT-07	Envio de alerta por SMS	SMS enviado
CT-08	Consulta do Dashboard e Histórico	Informação apresentada corretamente
NT-01	Palavra-passe incorreta	Mensagem de erro apresentada
NT-02	Código OTP incorreto	Acesso recusado
NT-03	Código OTP expirado	Código rejeitado
NT-04	Biometria desativada	Login biométrico bloqueado
API-01	Endpoint /login	Resposta HTTP correta

ID	Caso de Teste	Resultado Esperado
API-02	Endpoint /validate-otp	Validação correta do OTP

11.4 Ambiente de Testes

Os testes foram executados utilizando as seguintes tecnologias:

- Frontend: HTML5, CSS3 e JavaScript
- Backend: Node.js e Express.js
- Base de Dados: Ficheiro JSON
- Framework de Testes: Playwright
- Integração Contínua (CI): GitHub Actions
- Entrega Contínua (CD): Render
- Controlo de Versões: Git e GitHub

11.5 Processo de Execução dos Testes

Sempre que uma alteração é enviada para o repositório GitHub (push ou pull request), é iniciado automaticamente o pipeline de Integração Contínua (CI), que executa as seguintes etapas:

1. Obter o código mais recente do repositório.
2. Instalar as dependências do projeto.
3. Iniciar automaticamente o servidor Node.js.
4. Aguardar que a aplicação esteja disponível.
5. Executar todos os testes automatizados com Playwright.
6. Gerar o relatório de execução dos testes.
7. Caso todos os testes sejam aprovados, a aplicação é automaticamente atualizada na plataforma Render através do processo de Entrega Contínua (CD).

Este processo garante que apenas versões testadas e validadas são disponibilizadas em produção, reduzindo significativamente o risco de introdução de erros no sistema e aumentando a fiabilidade da aplicação.

8.BDD

A adoção da metodologia **Behavior-Driven Development (BDD)** proporcionou diversas vantagens ao desenvolvimento do projeto **Banco Digital Kriolu**, destacando-se:

- Especificação clara e compreensível dos requisitos funcionais.
- Melhor comunicação entre os membros da equipa de desenvolvimento e testes.
- Maior rastreabilidade entre requisitos, cenários e casos de teste.
- Automatização eficiente dos cenários através do Playwright.
- Detecção precoce de falhas durante o processo de desenvolvimento.
- Facilidade de manutenção e evolução dos testes à medida que novas funcionalidades são adicionadas.
- Integração direta com o **pipeline de Integração Contínua e Entrega Contínua (CI/CD)**, garantindo que todos os cenários são executados automaticamente sempre que ocorre uma alteração no código.

A utilização do **BDD** contribuiu para tornar o processo de validação do sistema mais organizado, documentado e alinhado com os requisitos do projeto, assegurando que as funcionalidades implementadas correspondem ao comportamento esperado pelos utilizadores e promovendo uma maior qualidade e fiabilidade da aplicação.

8.1 Cenários Gherkin

Cenário 1 – Login com sucesso

Funcionalidade: Login do utilizador

Cenário: Login com credenciais válidas

Dado que o utilizador possui uma conta ativa

Quando introduzir o nome de utilizador e a palavra-passe corretos

Então o sistema deve permitir o acesso à autenticação em dois fatores (2FA)

Cenário 2 – Validação do código OTP

Funcionalidade: Validação do OTP

Cenário: OTP válido

Dado que o utilizador recebeu um código **OTP**

Quando introduzir o código correto dentro do prazo de validade

Então o sistema deve permitir o acesso à conta

Cenário 3 – OTP expirado

Funcionalidade: Validação do OTP

Cenário: OTP expirado

Dado que o código OTP expirou

Quando o utilizador tentar utilizá-lo

Então o sistema deve rejeitar o código e solicitar um novo OTP

Cenário 4 – Recuperação da conta

Cenário: Solicitar recuperação da conta

Dado que o utilizador não consegue aceder à sua conta

Quando solicitar a recuperação da conta

Então a recuperação da conta deve ser efetuada presencialmente numa agência do Banco Digital Kriolu .

Cenário 5 – Autenticação biométrica

Funcionalidade: Autenticação biométrica

Cenário: Login com biometria

Dado que o dispositivo suporta autenticação biométrica

Quando o utilizador utilizar a impressão digital

Então o sistema deve autenticar o utilizador com sucesso

Cenário 6 – Acessos por OTP

Funcionalidade: Escolha do método **OTP**

Cenário: Receber **OTP** por **Email**

Dado que o utilizador pretende autenticar-se

Quando seleccionar **Email**

Então o sistema envia o código **OTP** para o e-mail indicad

9.CASOS DE TESTE

Os **casos de teste** foram elaborados com o objetivo de verificar se todas as funcionalidades do sistema **Banco Digital Kriolu** cumprem os requisitos **funcionais** e de **segurança** definidos. A sua conceção baseou-se em cenários de utilização reais, abrangendo tanto situações de funcionamento normal como condições de **erro** e **exceção**.

Para garantir uma cobertura abrangente, os testes foram organizados em três categorias: testes funcionais, responsáveis pela validação das funcionalidades principais; testes negativos, destinados à verificação do comportamento do sistema perante entradas inválidas ou situações inesperadas; e testes de **API**, que validam o correto funcionamento dos serviços disponibilizados pelo servidor.

Todos os casos de teste foram automatizados com o Playwright e executados no pipeline de Integração Contínua (**CI**) através do **GitHub Actions**, permitindo a validação automática da aplicação sempre que são realizadas alterações ao código-fonte.

A Tabela seguinte apresenta um resumo dos principais casos de teste implementados:

ID	Caso de Teste	Objetivo	Resultado Esperado
CT-01	Login com credenciais válidas	Validar a autenticação do utilizador	Login efetuado com sucesso
CT-02	Login com autenticação OTP	Validar o processo de autenticação em dois fatores	Acesso concedido após validação do OTP
CT-03	Login biométrico	Verificar a autenticação biométrica simulada	Utilizador autenticado com sucesso
CT-04	Logout	Confirmar o encerramento da sessão	Sessão terminada corretamente
CT-05	Dispositivo confiável	Validar o registo de dispositivos confiáveis	Dispositivo registado com sucesso
CT-06	Alerta por Email	Verificar o envio de alerta de segurança por email	Alerta enviado corretamente
CT-07	Alerta por SMS	Verificar o envio de alerta de segurança por SMS	Alerta enviado corretamente

ID	Caso de Teste	Objetivo	Resultado Esperado
CT-08	Dashboard e Histórico	Confirmar a apresentação das informações do utilizador	Dados e histórico apresentados corretamente
NT-01	Palavra-passe incorreta	Validar a rejeição de credenciais inválidas	Mensagem de erro apresentada
NT-02	Código OTP incorreto	Verificar a validação do código OTP	Acesso negado
NT-03	Código OTP expirado	Confirmar o tratamento de códigos expirados	Mensagem de OTP expirado
NT-04	Biometria desativada	Validar o bloqueio da autenticação biométrica	Acesso recusado
API-01	Endpoint /login	Validar o serviço de autenticação	Resposta HTTP correta
API-02	Endpoint /validate-otp	Validar o serviço de confirmação do OTP	Autenticação confirmada

Os resultados obtidos demonstraram que todos os cenários definidos foram executados com sucesso, evidenciando a conformidade do sistema com os requisitos funcionais e de segurança estabelecidos. A automatização destes casos de teste contribui para aumentar a fiabilidade do processo de desenvolvimento, permitindo identificar rapidamente eventuais regressões e garantindo a qualidade contínua da aplicação.

10.TESTES API

Os testes de API tiveram como objetivo validar o correto funcionamento dos serviços REST disponibilizados pelo backend do sistema Banco Digital Kriolu, garantindo que cada endpoint responde conforme os requisitos funcionais e de segurança definidos. Estes testes verificam não apenas as respostas devolvidas pelo servidor, mas também a integridade dos dados, os códigos de estado HTTP e o comportamento do sistema perante diferentes cenários de utilização.

A automatização dos testes foi realizada com o Playwright, recorrendo às funcionalidades de requisições HTTP (APIRequestContext), permitindo validar diretamente os serviços da API sem depender da interface gráfica. Esta abordagem torna os testes mais rápidos, fiáveis e adequados para integração contínua (CI/CD).

Foram desenvolvidos os seguintes testes de API:

ID	Endpoint	Objetivo	Resultado Esperado
API-01	POST /login	Validar a autenticação do utilizador com credenciais válidas e inválidas.	O sistema autentica o utilizador válido e rejeita credenciais incorretas com o respetivo código de erro.
API-02	POST /send-otp	Verificar a geração e envio do código OTP para autenticação em dois fatores.	O servidor gera um código OTP válido e responde com sucesso.
API-03	POST /validate-otp	Validar o código OTP enviado ao utilizador.	O acesso é autorizado quando o OTP é válido e rejeitado quando é inválido ou expirado.
API-04	POST /logout	Confirmar o encerramento da sessão do utilizador.	A sessão é terminada corretamente e registada no histórico.
API-05	GET /history/:username	Verificar a consulta do histórico de atividades do utilizador.	O sistema devolve todas as operações registadas para o utilizador autenticado.
API-06	GET /user/:username	Validar a obtenção dos dados do utilizador.	A API retorna corretamente as informações do utilizador solicitado.

ID	Endpoint	Objetivo	Resultado Esperado
API-07	POST /biometric-login	Testar a autenticação biométrica simulada.	O acesso é permitido apenas quando a biometria está ativada.
API-08	POST /trust-device	Validar o registo de dispositivos confiáveis.	O dispositivo é registado com sucesso para futuras autenticações.

Os resultados obtidos demonstraram que todos os endpoints implementados responderam conforme o comportamento esperado, validando corretamente os dados recebidos, aplicando as regras de segurança definidas e devolvendo os códigos HTTP apropriados. A execução automática destes testes no pipeline de Integração Contínua (CI) garante que qualquer alteração ao código é previamente validada antes de ser integrada na versão principal do projeto, contribuindo para a estabilidade, qualidade e fiabilidade do sistema.

11.GESTÃO DE DEFEITOS

A gestão de defeitos constituiu uma etapa essencial do processo de garantia da qualidade do sistema **Banco Digital Kriolu**, permitindo **identificar**, **registar**, **corrigir** e **validar** todas as não conformidades detetadas durante o desenvolvimento e a execução dos testes. O objetivo foi assegurar que os problemas encontrados fossem tratados de forma sistemática, reduzindo o risco de falhas na versão final da aplicação.

Durante a execução dos **testes funcionais**, de **API** e de integração contínua (**CI**), cada defeito identificado foi analisado quanto à sua causa, impacto e prioridade de correção. Após a implementação das respetivas correções, os testes foram novamente executados para confirmar a resolução do problema e garantir que as alterações não introduziram novos defeitos no sistema.

Entre os principais defeitos identificados e corrigidos destacam-se:

ID	Defeito Identificado	Correção Implementada	Estado
DF-01	Falha na autenticação com OTP inválido ou expirado	Implementação da validação da validade temporal e verificação do código OTP	Corrigido
DF-02	Conta não era bloqueada após múltiplas tentativas de login incorretas	Implementação do bloqueio automático após três tentativas falhadas	Corrigido
DF-03	Testes Playwright falhavam no GitHub Actions devido à URL fixa (127.0.0.1:5500)	Utilização da variável de ambiente BASE_URL, permitindo execução local e no pipeline CI	Corrigido
DF-04	Pipeline CI executava os testes antes de o servidor estar disponível	Configuração do arranque do servidor e sincronização através da ferramenta wait-on	Corrigido
DF-05	Erro no acesso às páginas da aplicação durante o deploy	Configuração correta do Express para servir os ficheiros estáticos da pasta <u>frontend</u>	Corrigido

ID	Defeito Identificado	Correção Implementada	Estado
DF-06	Página intermédia de autenticação (2FA) não era encontrada após o login no ambiente de produção	Ajuste das rotas e do processo de deploy da aplicação	Corrigido

A gestão estruturada destes defeitos permitiu aumentar significativamente a robustez e a estabilidade do sistema. A integração dos testes automatizados no pipeline de **Integração Contínua (CI)** garantiu ainda que todas as alterações ao código fossem automaticamente verificadas antes da integração na versão principal do projeto, reduzindo a probabilidade de regressões e assegurando a manutenção da qualidade do software ao longo do seu ciclo de desenvolvimento.

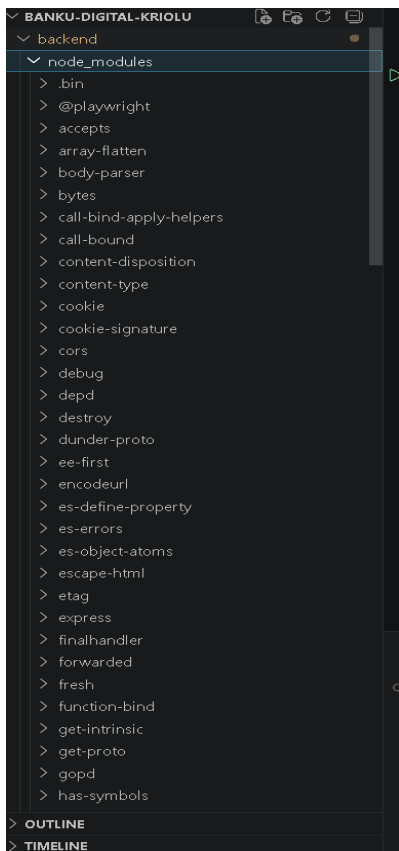
12.AUTOMAÇÃO DE TESTES

A automação de testes desempenhou um papel fundamental no processo de validação da qualidade do sistema **Banco Digital Kriolu**, permitindo a execução consistente, rápida e repetível dos cenários funcionais e das **APIs** desenvolvidas. A sua implementação teve como principal objetivo reduzir a intervenção manual, aumentar a cobertura dos testes e garantir que novas alterações ao sistema não introduzissem regressões.

Para a automação dos testes foi utilizada a ferramenta Playwright, responsável pela validação da interface gráfica (**Frontend**) e da interação do utilizador com o sistema, simulando cenários reais de utilização. Paralelamente, foram desenvolvidos testes para os serviços **REST API**, assegurando o correto funcionamento dos principais endpoints responsáveis pela autenticação, validação do código **OTP**, gestão de utilizadores e demais funcionalidades do sistema.

A execução automática dos testes foi integrada num pipeline de Integração Contínua (**CI**) através do **GitHub Actions**. Sempre que uma nova alteração é enviada para o repositório (push) ou é criada uma **Pull Request**, o pipeline é iniciado automaticamente, realizando as seguintes etapas:

- Configuração do ambiente de execução (Node.js);



- Instalação automática das dependências do projeto;
- Inicialização do servidor da aplicação;

The screenshot shows a VS Code editor with a file named `server.js` open. The code is written in JavaScript and defines an Express.js server. It includes CORS, JSON parsing, and file system operations to load and save users. The server is configured to run on port 3000. The terminal at the bottom shows the command `node server.js` being executed, and the output indicates that the server is running on `http://localhost:3000`.

```

backend > JS server.js > ...
6
7   app.use(cors());
8   app.use(express.json());
9
10  const PORT = 3000;
11
12  // =====
13  // Ler utilizadores
14  // =====
15
16  function carregarUtilizadores() {
17    return JSON.parse(fs.readFileSync("users.json", "utf8"));
18  }
19
20  // =====
21  // Guardar utilizadores
22  // =====
23
24  function guardarUtilizadores(users) {
25    fs.writeFileSync(
26      "users.json",
27      JSON.stringify(users, null, 2)
28    );
29  }
30
31  // =====
32  // Página inicial
33  // =====
34
35  app.get("/", (req, res) => {
36    res.send("Banco Digital Kriolu API");
37  });

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE PLAYWRIGHT

PS C:\Users\kalve\Downloads\ENI\3ºAno\2º Semestre\Teste Qualidade e Software\Banku-Digital-Kriolu> cd backend
 PS C:\Users\kalve\Downloads\ENI\3ºAno\2º Semestre\Teste Qualidade e Software\Banku-Digital-Kriolu\backend> node server.js
 Servidor iniciado em http://localhost:3000

- Execução dos testes automatizados com Playwright;

The screenshot shows a VS Code editor with a file named `api-login.spec.js` open. The code is written in JavaScript and defines a Playwright test for the login endpoint. The test uses `request.post` to send a POST request to `http://localhost:3000/login` with a JSON body containing `username: 'admin'` and `password: 'Admin123@kalves'`. The test expects a 200 status code and a JSON response with a message. The terminal at the bottom shows the command `node server.js` being executed, and the output indicates that the server is running on `http://localhost:3000`.

```

backend > tests > JS api-login.spec.js > ...
1   const { test, expect } = require('@playwright/test');
2
3   test('API-01 - POST /login', async ({ request }) => {
4
5     const resposta = await request.post(
6       'http://localhost:3000/login',
7       {
8         data: {
9           username: 'admin',
10          password: 'Admin123@kalves'
11        }
12      });
13
14     expect(resposta.status()).toBe(200);
15
16     const dados = await resposta.json();
17
18     expect(dados.message)
19       .toBe('Login efetuado com sucesso.');
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE PLAYWRIGHT

PS C:\Users\kalve\Downloads\ENI\3ºAno\2º Semestre\Teste Qualidade e Software\Banku-Digital-Kriolu> cd backend
 PS C:\Users\kalve\Downloads\ENI\3ºAno\2º Semestre\Teste Qualidade e Software\Banku-Digital-Kriolu\backend> node server.js
 Servidor iniciado em http://localhost:3000

- Geração do relatório de execução dos testes;

The screenshot shows a test execution report interface. At the top, there is a search bar labeled 'Search tests'. To the right, there are summary statistics: 'All 14', 'Passed 14', 'Failed 0', 'Flaky 0', and 'Skipped 0'. Below this, the date and time '7/8/2026, 12:21:47 AM' and the total time 'Total time: 43.5s' are displayed. The main content is a list of test suites, each with a dropdown arrow and a list of test cases. Each test case is marked with a green checkmark and includes its name and duration.

Test Suite	Test Case	Duration
api-login.spec.js	API-01 - POST /login	26ms
api-validate-otp.spec.js	API-02 - POST /validate-otp	14ms
biometria-desativada.spec.js	NT-04 - Biometria desativada	3.5s
biometria.spec.js	CT-03 - Login Biométrico	668ms
dashboard.spec.js	CT-08 - Dashboard + Histórico	810ms
dispositivo.spec.js	CT-05 - Dispositivo Confiável	807ms
email.spec.js	CT-06 - Alerta por Email	794ms

- Validação da qualidade antes da integração das alterações.

Esta abordagem permite identificar falhas de forma imediata, garantindo que apenas versões do sistema que satisfaçam todos os testes automatizados sejam consideradas estáveis.

A automatização implementada contribuiu significativamente para aumentar a fiabilidade do sistema, reduzir o tempo de validação das funcionalidades e apoiar uma estratégia de desenvolvimento contínuo baseada em boas práticas de Engenharia de Software e Garantia da Qualidade

Foram automatizados:

- Login
- OTP
- Logout
- Dashboard
- Biometria
- Email
- SMS

- API
- Casos negativos

Total de testes automatizados:

14 testes

Resultado final:

14/14 testes aprovados.

Objetivo

Verificar o comportamento do sistema quando a biometria está desativada.

Explicação

Primeiro o teste envia um pedido para o servidor para desativar a biometria.

```
await request.post(
  "http://localhost:3000/qa/biometria",
  {
    data: {
      username:"admin",
      enabled:false
    }
  });
```

Depois abre novamente a página de login.

```
await page.goto(`${BASE_URL}/login.html`);
```

Clica no botão de autenticação biométrica.

```
await page.click("#btnBiometria");
```

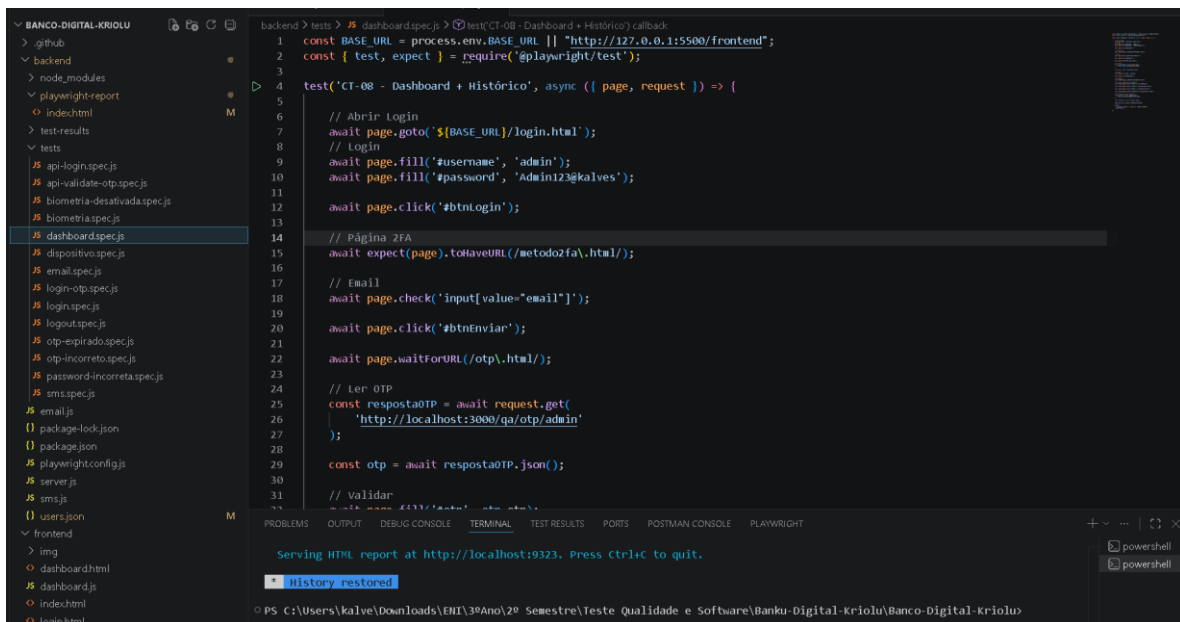
Por fim verifica se aparece a mensagem correta.

```
await expect(page.locator("#mensagem"))
```

```
.toContainText("Biometria não ativada.");
```

Assim garante que o sistema impede o login biométrico quando esta funcionalidade está desligada.

Dashboard.spec.js



```
1 const BASE_URL = process.env.BASE_URL || "http://127.0.0.1:5500/frontend";
2 const { test, expect } = require("@playwright/test");
3
4 test('CT-08 - Dashboard + Histórico', async ({ page, request }) => {
5
6     // Abrir Login
7     await page.goto(`${BASE_URL}/login.html`);
8     // Login
9     await page.fill('#username', 'admin');
10    await page.fill('#password', 'Admin123@kalves');
11
12    await page.click('#btnLogin');
13
14    // Página 2FA
15    await expect(page).toHaveURL(/metodo2fa\\.html/);
16
17    // Email
18    await page.check('input[value="email"]');
19
20    await page.click('#btnEnviar');
21
22    await page.waitForURL(/otp\\.html/);
23
24    // Ler OTP
25    const respostaOTP = await request.get(
26        'http://localhost:3000/qa/otp/admin'
27    );
28
29    const otp = await respostaOTP.json();
30
31    // Validar
```

Dashboard.spec.js

Objetivo

Verificar se, após o login e a autenticação por OTP, o utilizador consegue entrar corretamente no Dashboard.

Explicação

Este teste simula todo o processo de autenticação de um utilizador.

Primeiro abre a página de login.

```
await page.goto(`${BASE_URL}/login.html`);
```

Depois introduz as credenciais.

```
await page.fill("#username", "admin");
```

```
await page.fill("#password", "Admin123@kalves");
```

Efetua o login.

```
await page.click("#btnLogin");
```

Confirma que foi redirecionado para a página de autenticação de dois fatores.

```
await expect(page).toHaveURL(/metodo2fa\\.html/);
```

Seleciona o método Email.

```
await page.check('input[value="email"]');
```

Solicita o envio do código OTP.

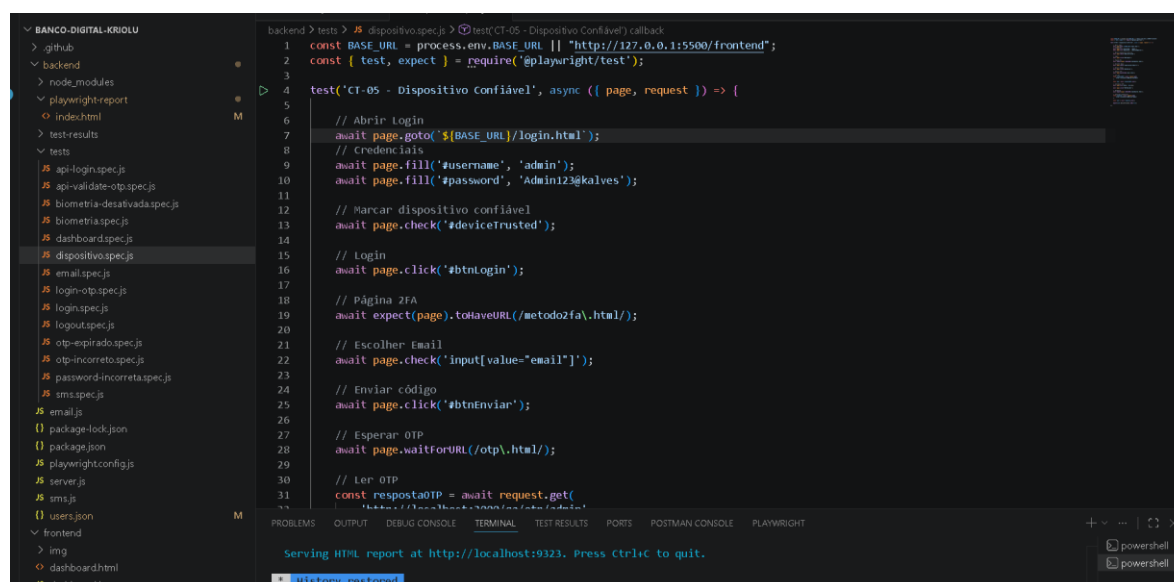
```
await page.click("#btnEnviar");
```

O teste lê automaticamente o OTP gerado pelo servidor.

```
const respostaOTP = await request.get("http://localhost:3000/qa/otp/admin");
```

Depois valida o código e confirma que o Dashboard foi aberto com sucesso.

Dispositivo.spec.js



```
back&end > tests > # dispositivo.spec.js > test('CT-05 - Dispositivo Confiável') callback
1  const BASE_URL = process.env.BASE_URL || "http://127.0.0.1:5500/frontend";
2  const { test, expect } = require('@playwright/test');
3
4  test('CT-05 - Dispositivo Confiável', async ({ page, request }) => {
5
6      // Abrir Login
7      await page.goto(`${BASE_URL}/login.html`);
8      // Credenciais
9      await page.fill('#username', 'admin');
10     await page.fill('#password', 'Admin123@kalves');
11
12     // Marcar dispositivo confiável
13     await page.check('#deviceTrusted');
14
15     // Login
16     await page.click('#btnLogin');
17
18     // Página 2FA
19     await expect(page).toHaveURL(/metodo2fa.html/);
20
21     // Escolher Email
22     await page.check('input[value="email"]');
23
24     // Enviar código
25     await page.click('#btnEnviar');
26
27     // Esperar OTP
28     await page.waitForURL(/otp.html/);
29
30     // Ler OTP
31     const respostaOTP = await request.get(
32         "http://localhost:3000/qa/otp/admin"
33     );
34
35     // Validar código
36     await page.fill('#codigo', respostaOTP);
37     await page.click('#btnValidar');
38
39     // Verificar se o dashboard foi aberto
40     await expect(page).toHaveURL(/dashboard.html/);
41
42     // Encerrar teste
43     await page.close();
44 });
```

Objetivo

Validar o funcionamento do dispositivo confiável.

Explicação

O teste abre a página de login.

```
await page.goto(`${BASE_URL}/login.html`);
```

Introduz as credenciais do utilizador.

```
await page.fill("#username", "admin");
```

```
await page.fill("#password", "Admin123@kalves");
```

Marca a opção de dispositivo confiável.

```
await page.check("#deviceTrusted");
```

Realiza o login.

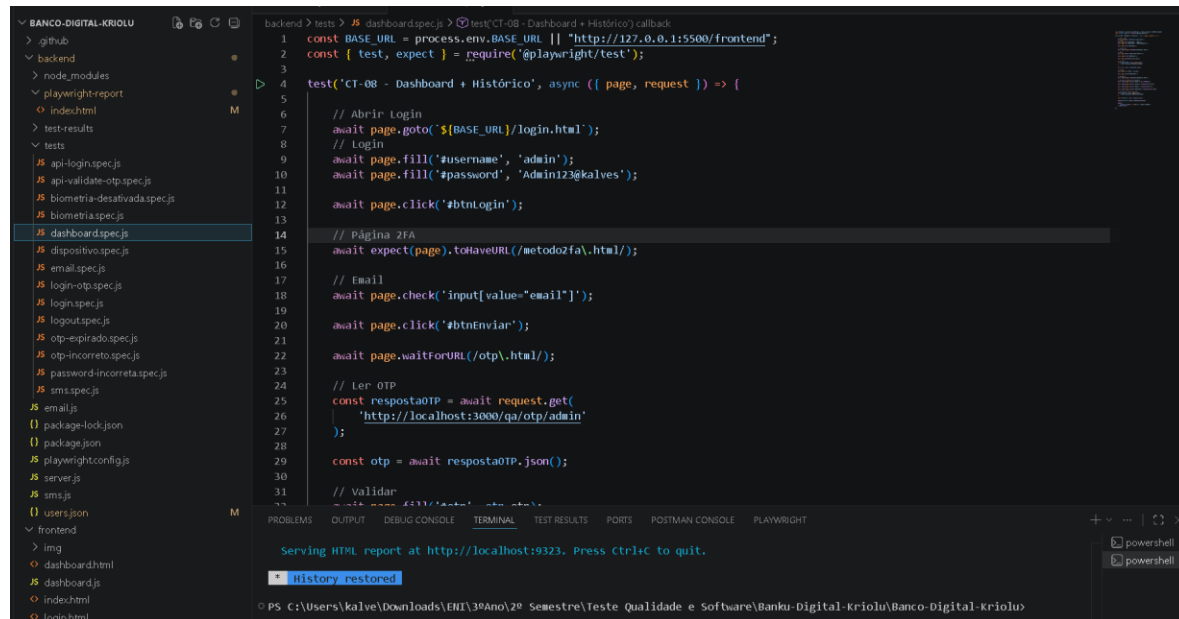
```
await page.click("#btnLogin");
```

Depois completa a autenticação OTP.

Finalmente confirma que o utilizador entra corretamente no Dashboard.

Este teste comprova que o sistema reconhece um dispositivo previamente autorizado.

Dashboard.spec.js



```
1 const BASE_URL = process.env.BASE_URL || "http://127.0.0.1:5500/frontend";
2 const { test, expect } = require("@playwright/test");
3
4 test('CT-08 - Dashboard + Histórico', async ({ page, request }) => {
5
6   // Abrir Login
7   await page.goto(`${BASE_URL}/login.html`);
8   // Login
9   await page.fill('#username', 'admin');
10  await page.fill('#password', 'Admin123@kalves');
11
12  await page.click('#btnLogin');
13
14  // Página 2FA
15  await expect(page).toHaveURL(/metodo2fa.html/);
16
17  // Email
18  await page.check('input[value="email"]');
19
20  await page.click('#btnEnviar');
21
22  await page.waitForURL(/otp.html/);
23
24  // Ler OTP
25  const respostaOTP = await request.get(
26    'http://localhost:3000/qa/otp/admin'
27  );
28
29  const otp = await respostaOTP.json();
30
31  // Validar
32  await page.click('#btnValidar');
```

Objetivo

Verificar se, após o login e a autenticação por OTP, o utilizador consegue entrar corretamente no Dashboard.

Explicação

Este teste simula todo o processo de autenticação de um utilizador.

Primeiro abre a página de login.

```
await page.goto(`${BASE_URL}/login.html`);
```

Depois introduz as credenciais.

```
await page.fill("#username", "admin");
```

```
await page.fill("#password", "Admin123@kalves");
```

Efetua o login.

```
await page.click("#btnLogin");
```

Confirma que foi redirecionado para a página de autenticação de dois fatores.

```
await expect(page).toHaveURL(/metodo2fa\.html/);
```

Seleciona o método Email.

```
await page.check('input[value="email"]');
```

Solicita o envio do código OTP.

```
await page.click("#btnEnviar");
```

O teste lê automaticamente o OTP gerado pelo servidor.

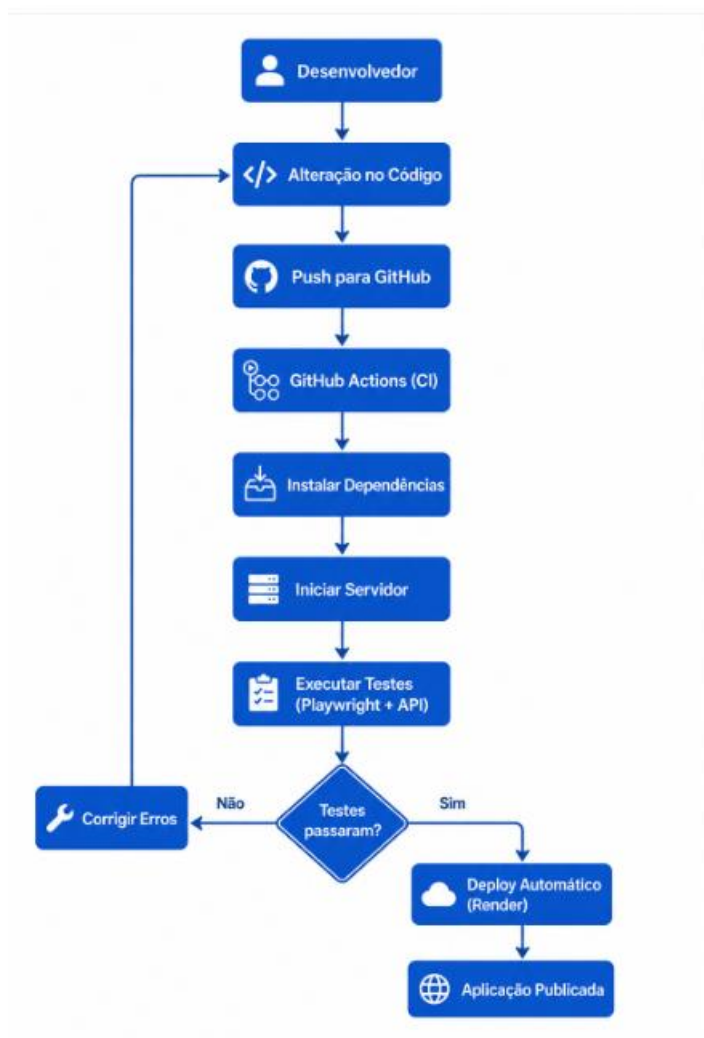
```
const respostaOTP = await request.get("http://localhost:3000/qa/otp/admin");
```

13. Pipeline CI/CD

O projeto Banco Digital Kriolu implementa um pipeline CI/CD (Continuous Integration / Continuous Delivery) para automatizar o processo de validação e publicação da aplicação. Sempre que uma alteração é enviada para o repositório GitHub, o GitHub Actions inicia automaticamente o pipeline, instalando as dependências, iniciando o servidor e executando todos os testes automatizados desenvolvidos com Playwright.

Caso todos os testes sejam concluídos com sucesso, a aplicação é automaticamente publicada na plataforma Render, garantindo que apenas versões testadas e estáveis são disponibilizadas aos utilizadores. Se algum teste falhar, o processo é interrompido, impedindo a publicação de versões com erros.

A utilização do pipeline CI/CD permite aumentar a qualidade do software, reduzir erros em produção, acelerar o processo de desenvolvimento e assegurar que todas as alterações passam por uma validação automática antes de serem disponibilizadas. Esta abordagem segue as boas práticas da Engenharia de Software, promovendo um desenvolvimento mais seguro, eficiente e fiável.



14.MÉTRICAS DE QUALIDADE

As métricas de qualidade foram utilizadas para avaliar o desempenho, a fiabilidade e a conformidade do sistema Banco Digital Kriolu em relação aos requisitos definidos. A análise destas métricas permitiu medir objetivamente a eficácia dos testes realizados e verificar o nível de qualidade alcançado pela aplicação antes da sua disponibilização.

Durante o desenvolvimento do projeto, foram monitorizados indicadores relacionados com a execução dos testes, deteção de defeitos e estabilidade da aplicação. A automatização dos testes com Playwright, integrada no pipeline CI/CD através do GitHub Actions, possibilitou a obtenção contínua destes indicadores a cada nova alteração ao código.

Principais Métricas Avaliadas:

Métrica	Resultado Obtido
Casos de teste implementados	14
Casos de teste executados	14
Casos de teste aprovados	14
Casos de teste reprovados	0
Taxa de sucesso dos testes	100%
Testes automatizados	100%
Testes de Interface (UI)	12
Testes de API	2
Pipeline CI executado	Sim

Análise dos Resultados

Os resultados obtidos demonstram que o Banco Digital Kriolu atingiu um elevado nível de qualidade, evidenciado pela aprovação de 100% dos casos de teste e pela execução bem-sucedida do pipeline de Integração Contínua (CI). A automatização dos testes garantiu a validação consistente das principais funcionalidades do sistema, enquanto a integração com o Render assegurou a Entrega Contínua (CD), publicando automaticamente apenas versões aprovadas. De forma geral, os indicadores confirmam a fiabilidade, estabilidade e conformidade da aplicação, comprovando a eficácia da estratégia de testes e das práticas de qualidade adotadas durante o desenvolvimento do projeto.

15. CONCLUSÃO

O desenvolvimento do Banco Digital Kriolu permitiu aplicar, de forma integrada, os principais conceitos de Engenharia de Software, Qualidade de Software e Automação de Testes, resultando numa aplicação web segura, funcional e alinhada com as boas práticas de desenvolvimento moderno.

Ao longo do projeto, foram implementadas funcionalidades essenciais, como autenticação de utilizadores, autenticação em dois fatores (OTP), login biométrico, gestão de dispositivos confiáveis, histórico de operações e mecanismos de alerta por email e SMS. Estas funcionalidades foram validadas através de um plano de testes estruturado, recorrendo à metodologia BDD (Behavior-Driven Development) e à automatização dos testes com Playwright, abrangendo tanto a interface do utilizador como os serviços de API.

A integração dos testes no pipeline de Integração Contínua e Entrega Contínua (CI/CD), utilizando GitHub Actions e Render, permitiu automatizar o processo de validação e publicação da aplicação, assegurando que apenas versões aprovadas pelos testes fossem disponibilizadas em produção. Esta abordagem contribuiu para aumentar a fiabilidade do sistema, reduzir o risco de regressões e melhorar a eficiência do processo de desenvolvimento.

Os resultados alcançados, com 100% dos testes automatizados aprovados, demonstram que a aplicação cumpre os requisitos funcionais e de qualidade estabelecidos, evidenciando a eficácia da estratégia de testes adotada.

Em conclusão, o projeto Banco Digital Kriolu constitui uma solução robusta e representa uma aplicação prática dos conhecimentos adquiridos na unidade curricular, demonstrando a importância da automação de testes, da integração contínua e da entrega contínua na produção de software de elevada qualidade, seguro, fiável e preparado para futuras evoluções.

17. REFERÊNCIAS BIBLIOGRÁFICAS

ISO/IEC 25010 – Systems and Software Quality Models.

- Playwright Documentation.
- Node.js Documentation.
- Express.js Documentation.
- GitHub Actions Documentation.
- Render Documentation.
- Material de apoio da Unidade Curricular de Testes e Qualidade de Software.